E-Commerce Database Design - Complete Documentation

1. Requirements Analysis

Functional Requirements

- Users must be able to register, login, and manage profiles
- Products must be organized into categories with inventory tracking
- Users can add products to shopping cart and place orders
- Orders must track payment status and delivery
- Users can review and rate products
- Users can maintain a wishlist
- Users can manage multiple delivery addresses
- Admin users can manage products, categories, users, and orders
- System must track order history and user activity

Non-Functional Requirements

- Data integrity and consistency (ACID properties)
- Scalability to handle large product catalogs and user bases
- Performance optimization through proper indexing
- Data security with proper access controls
- Support for concurrent transactions
- Efficient reporting and analytics queries
- Data normalization up to 3NF to eliminate redundancy

---

2. Entity Identification

Entity	Description
Users	Stores user account information, login credentials, and profile details
Roles	Defines user roles (Customer, Admin)
Categories	Product categories and subcategories
Products	Product details including name, description, price

Entity	Description
ProductImages	Multiple images for each product
Inventory	Stock tracking for products at warehouse/store level
ShoppingCart	Temporary storage of items before order placement
Orders	Order details including order date, status, total amount
OrderItems	Individual items in an order (line items)
Payments	Payment transaction details and status
Reviews	Product reviews and ratings by users
Wishlist	Products saved by users for future purchase
DeliveryAddresses	Multiple delivery addresses per user
OrderShipment	Shipment tracking information
Admins	Admin-specific information and permissions

---

3. Attributes & Data Types

Users Table

- user_id (INT, PK, AUTO_INCREMENT)
- email (VARCHAR(255), UNIQUE, NOT NULL)
- password_hash (VARCHAR(255), NOT NULL)
- first_name (VARCHAR(100), NOT NULL)
- last_name (VARCHAR(100), NOT NULL)
- phone (VARCHAR(20))
- date_of_birth (DATE)
- role_id (INT, FK → Roles.role_id)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP ON UPDATE)
- is_active (BOOLEAN, DEFAULT TRUE)

Roles Table

- role_id (INT, PK, AUTO_INCREMENT)
- role_name (VARCHAR(50), UNIQUE, NOT NULL) [Customer, Admin]

Categories Table

- category_id (INT, PK, AUTO_INCREMENT)

- category_name (VARCHAR(100), NOT NULL, UNIQUE)
- description (TEXT)
- parent_category_id (INT, FK → Categories.category_id) [For subcategories]
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

Products Table

- product_id (INT, PK, AUTO_INCREMENT)
- product_name (VARCHAR(255), NOT NULL)
- description (TEXT)
- category_id (INT, FK → Categories.category_id)
- price (DECIMAL(10, 2), NOT NULL)
- cost_price (DECIMAL(10, 2))
- sku (VARCHAR(100), UNIQUE, NOT NULL)
- is_active (BOOLEAN, DEFAULT TRUE)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP)

ProductImages Table

- image_id (INT, PK, AUTO_INCREMENT)
- product_id (INT, FK → Products.product_id)
- image_url (VARCHAR(500), NOT NULL)
- is_primary (BOOLEAN, DEFAULT FALSE)
- uploaded_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

Inventory Table

- inventory_id (INT, PK, AUTO_INCREMENT)
- product_id (INT, FK → Products.product_id, UNIQUE)
- quantity_in_stock (INT, NOT NULL, DEFAULT 0)
- reorder_level (INT)
- warehouse_location (VARCHAR(100))
- last_restocked (TIMESTAMP)

ShoppingCart Table

- cart_id (INT, PK, AUTO_INCREMENT)
- user_id (INT, FK → Users.user_id)
- product_id (INT, FK → Products.product_id)
- quantity (INT, NOT NULL, DEFAULT 1)

- added_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- UNIQUE KEY (user_id, product_id)

Orders Table

- order_id (INT, PK, AUTO_INCREMENT)
- user_id (INT, FK → Users.user_id)
- delivery_address_id (INT, FK → DeliveryAddresses.address_id)
- order_date (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- order_status (ENUM('Pending', 'Confirmed', 'Processing', 'Shipped', 'Delivered', 'Cancelled'), DEFAULT 'Pending')
- subtotal (DECIMAL(12, 2), NOT NULL)
- tax_amount (DECIMAL(10, 2), DEFAULT 0)
- shipping_cost (DECIMAL(10, 2), DEFAULT 0)
- discount_amount (DECIMAL(10, 2), DEFAULT 0)
- total_amount (DECIMAL(12, 2), NOT NULL)
- payment_status (ENUM('Pending', 'Completed', 'Failed'), DEFAULT 'Pending')

OrderItems Table

- order_item_id (INT, PK, AUTO_INCREMENT)
- order_id (INT, FK → Orders.order_id)
- product_id (INT, FK → Products.product_id)
- quantity (INT, NOT NULL)
- unit_price (DECIMAL(10, 2), NOT NULL)
- line_total (DECIMAL(12, 2), NOT NULL)

Payments Table

- payment_id (INT, PK, AUTO_INCREMENT)
- order_id (INT, FK → Orders.order_id)
- payment_method (ENUM('Credit Card', 'Debit Card', 'UPI', 'Wallet', 'Bank Transfer'), NOT NULL)
- amount (DECIMAL(12, 2), NOT NULL)
- payment_status (ENUM('Pending', 'Completed', 'Failed', 'Refunded'), DEFAULT 'Pending')
- transaction_id (VARCHAR(100), UNIQUE)
- payment_date (TIMESTAMP)
- gateway_response (JSON)

Reviews Table

- review_id (INT, PK, AUTO_INCREMENT)
- product_id (INT, FK → Products.product_id)

- user_id (INT, FK → Users.user_id)
- rating (INT, NOT NULL, CHECK (rating >= 1 AND rating <= 5))
- review_text (TEXT)
- is_verified_purchase (BOOLEAN, DEFAULT FALSE)
- helpful_count (INT, DEFAULT 0)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- updated_at (TIMESTAMP)
- UNIQUE KEY (product_id, user_id)

Wishlist Table

- wishlist_id (INT, PK, AUTO_INCREMENT)
- user_id (INT, FK → Users.user_id)
- product_id (INT, FK → Products.product_id)
- added_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)
- UNIQUE KEY (user_id, product_id)

DeliveryAddresses Table

- address_id (INT, PK, AUTO_INCREMENT)
- user_id (INT, FK → Users.user_id)
- address_type (ENUM('Home', 'Work', 'Other'), DEFAULT 'Home')
- street_address (VARCHAR(255), NOT NULL)
- city (VARCHAR(100), NOT NULL)
- state_province (VARCHAR(100))
- postal_code (VARCHAR(20), NOT NULL)
- country (VARCHAR(100), NOT NULL)
- is_default (BOOLEAN, DEFAULT FALSE)
- created_at (TIMESTAMP, DEFAULT CURRENT_TIMESTAMP)

OrderShipment Table

- shipment_id (INT, PK, AUTO_INCREMENT)
- order_id (INT, FK → Orders.order_id)
- tracking_number (VARCHAR(100), UNIQUE)
- carrier (VARCHAR(100))
- shipped_date (TIMESTAMP)
- estimated_delivery (DATE)
- actual_delivery (DATE)
- shipment_status (ENUM('Processing', 'In Transit', 'Out for Delivery', 'Delivered'), DEFAULT 'Processing')

Admins Table

- admin_id (INT, PK, AUTO_INCREMENT)
- user_id (INT, FK → Users.user_id, UNIQUE)
- permission_level (ENUM('SuperAdmin', 'Manager', 'Moderator'), DEFAULT 'Moderator')
- can_manage_products (BOOLEAN, DEFAULT FALSE)
- can_manage_users (BOOLEAN, DEFAULT FALSE)
- can_manage_orders (BOOLEAN, DEFAULT FALSE)
- can_view_reports (BOOLEAN, DEFAULT FALSE)
- last_login (TIMESTAMP)

---

4. Relationships

Relationship	Type	Description
Users → Roles	Many-to-One	Each user has one role; multiple users can have the same role
Products → Categories	Many-to-One	Each product belongs to one category; one category has many products
ProductImages → Products	Many-to-One	Multiple images per product; each image belongs to one product
Inventory → Products	One-to-One	Each product has one inventory record; each inventory record is for one product
ShoppingCart → Users	Many-to-One	One user can have many cart items; cart item belongs to one user
ShoppingCart → Products	Many-to-One	Multiple cart items can reference same product; each cart item is for one product
Orders → Users	Many-to-One	One user can place many orders; each order belongs to one user
Orders → DeliveryAddresses	Many-to-One	Order uses one address; user can have many addresses
OrderItems → Orders	Many-to-One	One order has many line items; each line item belongs to one order
OrderItems → Products	Many-to-One	Multiple order items can reference same product
Payments → Orders	Many-to-One	Each order can have multiple payment records; payment belongs to one order

Relationship	Type	Description
Reviews → Products	Many-to-One	One product has many reviews; each review is for one product
Reviews → Users	Many-to-One	One user can review many products; review is by one user
Wishlist → Users	Many-to-One	One user has many wishlist items; wishlist item belongs to one user
Wishlist → Products	Many-to-One	Multiple wishlist items can reference same product
DeliveryAddresses → Users	Many-to-One	One user has many delivery addresses; address belongs to one user
OrderShipment → Orders	Many-to-One	One order can have multiple shipments; shipment belongs to one order
Admins → Users	One-to-One	Admin record extends user record; each admin is associated with exactly one user

---

5. ER Diagram (PlantUML Format)

```
@startuml E-Commerce Database

entity Users {
  *user_id : INT <<PK>>
  email : VARCHAR(255) <<UNIQUE>>
  password_hash : VARCHAR(255)
  first_name : VARCHAR(100)
  last_name : VARCHAR(100)
  phone : VARCHAR(20)
  date_of_birth : DATE
  role_id : INT <<FK>>
  created_at : TIMESTAMP
  is_active : BOOLEAN
}

entity Roles {
  *role_id : INT <<PK>>
  role_name : VARCHAR(50) <<UNIQUE>>
}
```

```
entity Categories {
  *category_id : INT <<PK>>
  category_name : VARCHAR(100) <<UNIQUE>>
  description : TEXT
  parent_category_id : INT <<FK>>
}

entity Products {
  *product_id : INT <<PK>>
  product_name : VARCHAR(255)
  description : TEXT
  category_id : INT <<FK>>
  price : DECIMAL(10,2)
  sku : VARCHAR(100) <<UNIQUE>>
  is_active : BOOLEAN
}

entity ProductImages {
  *image_id : INT <<PK>>
  product_id : INT <<FK>>
  image_url : VARCHAR(500)
  is_primary : BOOLEAN
}

entity Inventory {
  *inventory_id : INT <<PK>>
  product_id : INT <<FK>> <<UNIQUE>>
  quantity_in_stock : INT
  reorder_level : INT
}

entity ShoppingCart {
  *cart_id : INT <<PK>>
  user_id : INT <<FK>>
  product_id : INT <<FK>>
  quantity : INT
}

entity Orders {
  *order_id : INT <<PK>>
  user_id : INT <<FK>>
  delivery_address_id : INT <<FK>>
  order_date : TIMESTAMP
  order_status : ENUM
```

```
  total_amount : DECIMAL(12,2)
  payment_status : ENUM
}

entity OrderItems {
  *order_item_id : INT <<PK>>
  order_id : INT <<FK>>
  product_id : INT <<FK>>
  quantity : INT
  unit_price : DECIMAL(10,2)
}

entity Payments {
  *payment_id : INT <<PK>>
  order_id : INT <<FK>>
  payment_method : ENUM
  amount : DECIMAL(12,2)
  payment_status : ENUM
  transaction_id : VARCHAR(100)
}

entity Reviews {
  *review_id : INT <<PK>>
  product_id : INT <<FK>>
  user_id : INT <<FK>>
  rating : INT
  review_text : TEXT
  is_verified_purchase : BOOLEAN
}

entity Wishlist {
  *wishlist_id : INT <<PK>>
  user_id : INT <<FK>>
  product_id : INT <<FK>>
}

entity DeliveryAddresses {
  *address_id : INT <<PK>>
  user_id : INT <<FK>>
  street_address : VARCHAR(255)
  city : VARCHAR(100)
  postal_code : VARCHAR(20)
  is_default : BOOLEAN
}
```

```
entity OrderShipment {
  *shipment_id : INT <<PK>>
  order_id : INT <<FK>>
  tracking_number : VARCHAR(100)
  carrier : VARCHAR(100)
  shipment_status : ENUM
}

entity Admins {
  *admin_id : INT <<PK>>
  user_id : INT <<FK>> <<UNIQUE>>
  permission_level : ENUM
  can_manage_products : BOOLEAN
  can_manage_orders : BOOLEAN
}

Users ||--o{ Roles : "has"
Categories ||--o{ Products : "contains"
Products ||--o{ ProductImages : "has"
Products ||--|| Inventory : "tracked by"
Users ||--o{ ShoppingCart : "adds to"
Products ||--o{ ShoppingCart : "in"
Users ||--o{ Orders : "places"
Orders ||--o{ OrderItems : "contains"
Products ||--o{ OrderItems : "in"
Orders ||--o{ Payments : "paid by"
Products ||--o{ Reviews : "reviewed in"
Users ||--o{ Reviews : "writes"
Users ||--o{ Wishlist : "has"
Products ||--o{ Wishlist : "in"
Users ||--o{ DeliveryAddresses : "has"
Orders ||--o{ DeliveryAddresses : "shipped to"
Orders ||--o{ OrderShipment : "tracked by"
Users ||--|| Admins : "assigned as"

@enduml
```

6. Normalization (Up to 3NF)

First Normal Form (1NF)

Goal: Eliminate repeating groups and ensure atomicity

Rules Applied:

- All attributes contain atomic (indivisible) values
- No repeating groups of columns
- All rows must have the same number of columns

What we did:

- Separated ProductImages into its own table (products could have multiple images)
- Created OrderItems table (orders can have multiple line items)
- Separated Reviews table (one product can have many reviews)
- Used ENUM for fixed values (payment_method, order_status)

Example: Instead of storing multiple images in one Product row like `images = ['img1.jpg', 'img2.jpg']`, we created ProductImages table with separate rows.

Second Normal Form (2NF)

Goal: Remove partial dependencies (non-key attributes must depend on the entire primary key)

Rules Applied:

- Must be in 1NF
- All non-key attributes must depend on the entire primary key

What we did:

- In OrderItems: unit_price and line_total depend on (order_id, product_id) combination, not just product_id, because price changes over time
- In ShoppingCart: quantity depends on (user_id, product_id), not just one
- Inventory: quantity_in_stock depends on the entire inventory_id (which is product_id)
- Separated line items from Orders because order details (unit price, quantity) depend on the specific order

Example: In OrderItems, we don't store product name or category repeatedly—they're only in Products table. Each row represents (order_id, product_id) combination with quantity and price specific to that order.

Third Normal Form (3NF)

Goal: Remove transitive dependencies (non-key attributes must not depend on other non-key attributes)

Rules Applied:

- Must be in 2NF
- No non-key attribute depends on another non-key attribute
- All transitive dependencies eliminated

What we did:

- In Products: We don't store category_name directly; we store category_id (FK). This prevents update anomalies if a category name changes.
- In Orders: We store user_id (FK) and delivery_address_id (FK), not user details or address details
- In Reviews: We store user_id and product_id (FKs), not user details or product details
- In OrderShipment: We store order_id (FK), not order details

Example: Instead of storing user's address details directly in Orders, we store delivery_address_id which references DeliveryAddresses table. If an address changes, it's updated in one place.

---

7. SQL Schema - CREATE TABLE Statements (PostgreSQL)

```sql
```

```sql
-- Create Roles Table
CREATE TABLE roles (
    role_id SERIAL PRIMARY KEY,
    role_name VARCHAR(50) UNIQUE NOT NULL
);

-- Create Users Table
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    date_of_birth DATE,
    role_id INT NOT NULL DEFAULT 1,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    FOREIGN KEY (role_id) REFERENCES roles(role_id)
);

-- Create Categories Table
CREATE TABLE categories (
    category_id SERIAL PRIMARY KEY,
    category_name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,
    parent_category_id INT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (parent_category_id) REFERENCES categories(category_id) ON DELETE SET NULL
);

-- Create Products Table
CREATE TABLE products (
    product_id SERIAL PRIMARY KEY,
    product_name VARCHAR(255) NOT NULL,
    description TEXT,
    category_id INT NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    cost_price DECIMAL(10, 2),
    sku VARCHAR(100) UNIQUE NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (category_id) REFERENCES categories(category_id) ON DELETE RESTRICT
);

-- Create ProductImages Table
CREATE TABLE product_images (
    image_id SERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    image_url VARCHAR(500) NOT NULL,
    is_primary BOOLEAN DEFAULT FALSE,
    uploaded_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);

-- Create Inventory Table
CREATE TABLE inventory (
    inventory_id SERIAL PRIMARY KEY,
    product_id INT UNIQUE NOT NULL,
    quantity_in_stock INT NOT NULL DEFAULT 0,
    reorder_level INT,
    warehouse_location VARCHAR(100),
    last_restocked TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);

-- Create ShoppingCart Table
CREATE TABLE shopping_cart (
    cart_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL DEFAULT 1,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, product_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);

-- Create DeliveryAddresses Table
CREATE TABLE delivery_addresses (
    address_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    address_type VARCHAR(50) DEFAULT 'Home',
    street_address VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
```

```
    state_province VARCHAR(100),
    postal_code VARCHAR(20) NOT NULL,
    country VARCHAR(100) NOT NULL,
    is_default BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- Create Orders Table
CREATE TABLE orders (
    order_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    delivery_address_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    order_status VARCHAR(50) DEFAULT 'Pending' CHECK (order_status IN ('Pending', 'Confirmed', 'Processing', 'Shipped'
    subtotal DECIMAL(12, 2) NOT NULL,
    tax_amount DECIMAL(10, 2) DEFAULT 0,
    shipping_cost DECIMAL(10, 2) DEFAULT 0,
    discount_amount DECIMAL(10, 2) DEFAULT 0,
    total_amount DECIMAL(12, 2) NOT NULL,
    payment_status VARCHAR(50) DEFAULT 'Pending' CHECK (payment_status IN ('Pending', 'Completed', 'Failed')),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE RESTRICT,
    FOREIGN KEY (delivery_address_id) REFERENCES delivery_addresses(address_id)
);

-- Create OrderItems Table
CREATE TABLE order_items (
    order_item_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10, 2) NOT NULL,
    line_total DECIMAL(12, 2) NOT NULL,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE RESTRICT
);

-- Create Payments Table
CREATE TABLE payments (
    payment_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    payment_method VARCHAR(50) NOT NULL CHECK (payment_method IN ('Credit Card', 'Debit Card', 'UPI', 'Wallet', 'B
    amount DECIMAL(12, 2) NOT NULL,
    payment_status VARCHAR(50) DEFAULT 'Pending' CHECK (payment_status IN ('Pending', 'Completed', 'Failed', 'Refun
```

```
    transaction_id VARCHAR(100) UNIQUE,
    payment_date TIMESTAMP,
    gateway_response JSONB,
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE
);

-- Create Reviews Table
CREATE TABLE reviews (
    review_id SERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    user_id INT NOT NULL,
    rating INT NOT NULL CHECK (rating >= 1 AND rating <= 5),
    review_text TEXT,
    is_verified_purchase BOOLEAN DEFAULT FALSE,
    helpful_count INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(product_id, user_id),
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- Create Wishlist Table
CREATE TABLE wishlist (
    wishlist_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    product_id INT NOT NULL,
    added_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(user_id, product_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (product_id) REFERENCES products(product_id) ON DELETE CASCADE
);

-- Create OrderShipment Table
CREATE TABLE order_shipment (
    shipment_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL,
    tracking_number VARCHAR(100) UNIQUE,
    carrier VARCHAR(100),
    shipped_date TIMESTAMP,
    estimated_delivery DATE,
    actual_delivery DATE,
    shipment_status VARCHAR(50) DEFAULT 'Processing' CHECK (shipment_status IN ('Processing', 'In Transit', 'Out for D
    FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE
```

```sql
);

-- Create Admins Table
CREATE TABLE admins (
    admin_id SERIAL PRIMARY KEY,
    user_id INT UNIQUE NOT NULL,
    permission_level VARCHAR(50) DEFAULT 'Moderator' CHECK (permission_level IN ('SuperAdmin', 'Manager', 'Modera
    can_manage_products BOOLEAN DEFAULT FALSE,
    can_manage_users BOOLEAN DEFAULT FALSE,
    can_manage_orders BOOLEAN DEFAULT FALSE,
    can_view_reports BOOLEAN DEFAULT FALSE,
    last_login TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

-- Create Indexes for Performance Optimization
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_role_id ON users(role_id);
CREATE INDEX idx_products_category_id ON products(category_id);
CREATE INDEX idx_products_sku ON products(sku);
CREATE INDEX idx_inventory_product_id ON inventory(product_id);
CREATE INDEX idx_shopping_cart_user_id ON shopping_cart(user_id);
CREATE INDEX idx_shopping_cart_product_id ON shopping_cart(product_id);
CREATE INDEX idx_orders_user_id ON orders(user_id);
CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_orders_order_status ON orders(order_status);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
CREATE INDEX idx_order_items_product_id ON order_items(product_id);
CREATE INDEX idx_payments_order_id ON payments(order_id);
CREATE INDEX idx_payments_transaction_id ON payments(transaction_id);
CREATE INDEX idx_reviews_product_id ON reviews(product_id);
CREATE INDEX idx_reviews_user_id ON reviews(user_id);
CREATE INDEX idx_wishlist_user_id ON wishlist(user_id);
CREATE INDEX idx_wishlist_product_id ON wishlist(product_id);
CREATE INDEX idx_delivery_addresses_user_id ON delivery_addresses(user_id);
CREATE INDEX idx_order_shipment_order_id ON order_shipment(order_id);
CREATE INDEX idx_order_shipment_tracking ON order_shipment(tracking_number);
```

---

8. Sample Data - INSERT Statements

```sql
```

```sql
-- Insert Roles
INSERT INTO roles (role_name) VALUES
('Customer'),
('Admin');

-- Insert Users
INSERT INTO users (email, password_hash, first_name, last_name, phone, date_of_birth, role_id) VALUES
('john.doe@email.com', 'hashed_password_1', 'John', 'Doe', '9876543210', '1990-05-15', 1),
('jane.smith@email.com', 'hashed_password_2', 'Jane', 'Smith', '9876543211', '1992-08-22', 1),
('admin@ecommerce.com', 'hashed_password_admin', 'Admin', 'User', '9876543212', '1985-03-10', 2),
('sarah.johnson@email.com', 'hashed_password_3', 'Sarah', 'Johnson', '9876543213', '1995-11-30', 1);

-- Insert Categories
INSERT INTO categories (category_name, description) VALUES
('Electronics', 'Electronic devices and gadgets'),
('Clothing', 'Apparel and fashion items'),
('Books', 'Physical and digital books'),
('Home & Garden', 'Home decor and garden supplies');

-- Insert Subcategories
INSERT INTO categories (category_name, description, parent_category_id) VALUES
('Mobile Phones', 'Smartphones and accessories', 1),
('Laptops', 'Computers and notebooks', 1),
('Men Clothing', 'Mens apparel', 2);

-- Insert Products
INSERT INTO products (product_name, description, category_id, price, cost_price, sku) VALUES
('iPhone 14 Pro', 'Latest Apple smartphone with advanced features', 5, 999.99, 700.00, 'SKU-IP14P-001'),
('Samsung Galaxy S23', 'Premium Android phone with 200MP camera', 5, 899.99, 600.00, 'SKU-SAM-S23-001'),
('Dell XPS 13', 'Ultra-portable laptop with powerful performance', 6, 1299.99, 900.00, 'SKU-DELL-XPS-001'),
('The Great Gatsby', 'Classic American novel by F. Scott Fitzgerald', 3, 9.99, 4.00, 'SKU-BOOK-GG-001'),
('Blue Denim Jeans', 'Comfortable and stylish mens jeans', 7, 49.99, 25.00, 'SKU-JEAN-BL-001');

-- Insert ProductImages
INSERT INTO product_images (product_id, image_url, is_primary) VALUES
(1, 'https://images.example.com/iphone14-1.jpg', TRUE),
(1, 'https://images.example.com/iphone14-2.jpg', FALSE),
(2, 'https://images.example.com/galaxy-s23-1.jpg', TRUE),
(3, 'https://images.example.com/dell-xps-1.jpg', TRUE),
(4, 'https://images.example.com/gatsby-1.jpg', TRUE),
(5, 'https://images.example.com/jeans-1.jpg', TRUE);

-- Insert Inventory
```

```
INSERT INTO inventory (product_id, quantity_in_stock, reorder_level, warehouse_location) VALUES
(1, 150, 20, 'Warehouse A - Rack 5'),
(2, 200, 30, 'Warehouse B - Rack 3'),
(3, 80, 10, 'Warehouse A - Rack 8'),
(4, 500, 50, 'Warehouse C - Rack 2'),
(5, 300, 40, 'Warehouse B - Rack 6');

-- Insert ShoppingCart Items
INSERT INTO shopping_cart (user_id, product_id, quantity) VALUES
(1, 1, 1),
(1, 5, 2),
(2, 3, 1);

-- Insert Delivery Addresses
INSERT INTO delivery_addresses (user_id, address_type, street_address, city, state_province, postal_code, country, is_defau
(1, 'Home', '123 Main Street', 'New York', 'NY', '10001', 'USA', TRUE),
(1, 'Work', '456 Business Ave', 'New York', 'NY', '10005', 'USA', FALSE),
(2, 'Home', '789 Oak Road', 'Los Angeles', 'CA', '90001', 'USA', TRUE),
(4, 'Home', '321 Elm Street', 'Chicago', 'IL', '60601', 'USA', TRUE);

-- Insert Orders
INSERT INTO orders (user_id, delivery_address_id, order_date, order_status, subtotal, tax_amount, shipping_cost, discount_
(1, 1, '2025-10-15 10:30:00', 'Delivered', 1049.98, 84.00, 10.00, 50.00, 1093.98, 'Completed'),
(2, 3, '2025-10-16 14:15:00', 'Shipped', 1299.99, 104.00, 15.00, 0.00, 1418.99, 'Completed'),
(1, 1, '2025-10-18 16:45:00', 'Processing', 99.98, 8.00, 5.00, 0.00, 112.98, 'Pending'),
(4, 4, '2025-10-19 09:20:00', 'Confirmed', 9.99, 0.80, 3.00, 0.00, 13.79, 'Completed');

-- Insert OrderItems
INSERT INTO order_items (order_id, product_id, quantity, unit_price, line_total) VALUES
(1, 1, 1, 999.99, 999.99),
(1, 5, 1, 49.99, 49.99),
(2, 3, 1, 1299.99, 1299.99),
(3, 5, 2, 49.99, 99.98),
(4, 4, 1, 9.99, 9.99);

-- Insert Payments
INSERT INTO payments (order_id, payment_method, amount, payment_status, transaction_id, payment_date) VALUES
(1, 'Credit Card', 1093.98, 'Completed', 'TXN-001-20251015', '2025-10-15 10:35:00'),
(2, 'Debit Card', 1418.99, 'Completed', 'TXN-002-20251016', '2025-10-16 14:20:00'),
(3, 'UPI', 112.98, 'Pending', 'TXN-003-20251018', NULL),
(4, 'Wallet', 13.79, 'Completed', 'TXN-004-20251019', '2025-10-19 09:25:00');

-- Insert Reviews
INSERT INTO reviews (product_id, user_id, rating, review_text, is_verified_purchase, helpful_count) VALUES
```

```
(1, 1, 5, 'Excellent phone! Great camera and performance. Highly recommended!', TRUE, 42),
(1, 2, 4, 'Good phone but battery life could be better.', FALSE, 15),
(3, 2, 5, 'Amazing laptop! Very fast and portable. Worth every penny!', TRUE, 28),
(4, 4, 5, 'Classic novel, beautifully written. A must-read!', TRUE, 8),
(5, 1, 4, 'Comfortable jeans, great fit and quality material.', TRUE, 12);

-- Insert Wishlist Items
INSERT INTO wishlist (user_id, product_id) VALUES
(1, 2),
(1, 4),
(2, 1),
(2, 5),
(4, 3);

-- Insert OrderShipment
INSERT INTO order_shipment (order_id, tracking_number, carrier, shipped_date, estimated_delivery, actual_delivery, shipm
(1, 'TRACK-20251015-001', 'FedEx', '2025-10-16 08:00:00', '2025-10-18', '2025-10-18', 'Delivered'),
(2, 'TRACK-20251016-002', 'UPS', '2025-10-17 10:00:00', '2025-10-20', NULL, 'In Transit'),
(3, 'TRACK-20251018-003', 'DHL', '2025-10-19 09:00:00', '2025-10-21', NULL, 'Processing'),
(4, 'TRACK-20251019-004', 'FedEx', '2025-10-20 14:00:00', '2025-10-22', NULL, 'Processing');

-- Insert Admin
INSERT INTO admins (user_id, permission_level, can_manage_products, can_manage_users, can_manage_orders, can_view
(3, 'SuperAdmin', TRUE, TRUE, TRUE, TRUE);
```

---

9. Example Queries

Query 1: Get all orders placed by a specific user with order details

```
sql
```

```sql
SELECT
    o.order_id,
    o.order_date,
    o.order_status,
    o.total_amount,
    o.payment_status,
    da.street_address,
    da.city,
    da.country
FROM orders o
JOIN delivery_addresses da ON o.delivery_address_id = da.address_id
WHERE o.user_id = 1
ORDER BY o.order_date DESC;
```

Query 2: Calculate total revenue from completed orders

sql

```sql
SELECT
    SUM(o.total_amount) as total_revenue,
    COUNT(o.order_id) as total_orders,
    AVG(o.total_amount) as average_order_value
FROM orders o
WHERE o.payment_status = 'Completed'
    AND o.order_date >= CURRENT_DATE - INTERVAL '30 days';
```

Query 3: Get top 5 best-selling products

sql

```sql
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    SUM(oi.quantity) as total_quantity_sold,
    SUM(oi.line_total) as total_revenue,
    ROUND(AVG(r.rating), 2) as average_rating
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
JOIN categories c ON p.category_id = c.category_id
LEFT JOIN reviews r ON p.product_id = r.product_id
WHERE o.payment_status = 'Completed'
GROUP BY p.product_id, p.product_name, c.category_name
ORDER BY total_quantity_sold DESC
LIMIT 5;
```

Query 4: Get all reviews for a product with reviewer details

```sql
SELECT
    r.review_id,
    r.rating,
    r.review_text,
    r.is_verified_purchase,
    r.helpful_count,
    r.created_at,
    u.first_name,
    u.last_name,
    u.email
FROM reviews r
JOIN users u ON r.user_id = u.user_id
WHERE r.product_id = 1
ORDER BY r.created_at DESC;
```

Query 5: Find products with low inventory (below reorder level)

```sql
```

```sql
SELECT
    p.product_id,
    p.product_name,
    p.sku,
    i.quantity_in_stock,
    i.reorder_level,
    i.warehouse_location,
    c.category_name
FROM products p
JOIN inventory i ON p.product_id = i.product_id
JOIN categories c ON p.category_id = c.category_id
WHERE i.quantity_in_stock <= i.reorder_level
    AND p.is_active = TRUE
ORDER BY i.quantity_in_stock ASC;
```

Query 6: Get user shopping cart with product details and total

sql

```sql
SELECT
    sc.cart_id,
    p.product_id,
    p.product_name,
    p.price,
    sc.quantity,
    (p.price * sc.quantity) as line_total,
    c.category_name
FROM shopping_cart sc
JOIN products p ON sc.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
WHERE sc.user_id = 1
ORDER BY sc.added_at DESC;
```

Query 7: Calculate monthly sales by category

sql

```sql
SELECT
    c.category_name,
    DATE_TRUNC('month', o.order_date) as month,
    COUNT(oi.order_item_id) as items_sold,
    SUM(oi.line_total) as category_revenue,
    ROUND(AVG(oi.unit_price), 2) as avg_price
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.payment_status = 'Completed'
GROUP BY c.category_id, c.category_name, DATE_TRUNC('month', o.order_date)
ORDER BY month DESC, category_revenue DESC;
```

Query 8: Get order details with all line items

sql

```sql
SELECT
    o.order_id,
    o.order_date,
    o.total_amount,
    o.order_status,
    u.first_name,
    u.last_name,
    u.email,
    oi.order_item_id,
    p.product_name,
    oi.quantity,
    oi.unit_price,
    oi.line_total
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.order_id = 1
ORDER BY oi.order_item_id;
```

Query 9: Get user's wishlist with product availability

```sql
SELECT
    w.wishlist_id,
    p.product_id,
    p.product_name,
    p.price,
    c.category_name,
    i.quantity_in_stock,
    CASE
        WHEN i.quantity_in_stock > 0 THEN 'In Stock'
        ELSE 'Out of Stock'
    END as availability,
    ROUND(AVG(r.rating), 2) as avg_rating
FROM wishlist w
JOIN products p ON w.product_id = p.product_id
JOIN categories c ON p.category_id = c.category_id
JOIN inventory i ON p.product_id = i.product_id
LEFT JOIN reviews r ON p.product_id = r.product_id
WHERE w.user_id = 1
GROUP BY w.wishlist_id, p.product_id, p.product_name, p.price, c.category_name, i.quantity_in_stock
ORDER BY w.added_at DESC;
```

Query 10: Get shipment tracking information for an order

```sql
```

```sql
SELECT
    o.order_id,
    o.order_date,
    os.shipment_id,
    os.tracking_number,
    os.carrier,
    os.shipment_status,
    os.shipped_date,
    os.estimated_delivery,
    os.actual_delivery,
    u.first_name,
    u.last_name,
    da.street_address,
    da.city,
    da.country
FROM order_shipment os
JOIN orders o ON os.order_id = o.order_id
JOIN users u ON o.user_id = u.user_id
JOIN delivery_addresses da ON o.delivery_address_id = da.address_id
WHERE o.order_id = 2;
```

Query 11: Get average rating and review count per product

```sql
```

```sql
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    COUNT(r.review_id) as review_count,
    ROUND(AVG(r.rating), 2) as average_rating,
    SUM(CASE WHEN r.rating = 5 THEN 1 ELSE 0 END) as five_star_count,
    SUM(CASE WHEN r.rating = 4 THEN 1 ELSE 0 END) as four_star_count,
    SUM(CASE WHEN r.rating = 3 THEN 1 ELSE 0 END) as three_star_count,
    SUM(CASE WHEN r.rating = 2 THEN 1 ELSE 0 END) as two_star_count,
    SUM(CASE WHEN r.rating = 1 THEN 1 ELSE 0 END) as one_star_count
FROM products p
JOIN categories c ON p.category_id = c.category_id
LEFT JOIN reviews r ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_name, c.category_name
HAVING COUNT(r.review_id) > 0
ORDER BY average_rating DESC, review_count DESC;
```

Query 12: Get admin permissions for a specific admin user

```sql
sql

SELECT
    a.admin_id,
    u.first_name,
    u.last_name,
    u.email,
    a.permission_level,
    a.can_manage_products,
    a.can_manage_users,
    a.can_manage_orders,
    a.can_view_reports,
    a.last_login
FROM admins a
JOIN users u ON a.user_id = u.user_id
WHERE a.user_id = 3;
```

---

10. Summary: Why This Database Design is Efficient for Real-World E-Commerce

1. Data Integrity & Consistency

- Foreign key constraints enforce referential integrity
- CHECK constraints validate data (e.g., ratings between 1-5, valid payment methods)
- UNIQUE constraints prevent duplicate entries (email, SKU, tracking numbers)
- ACID properties ensure reliable transactions

2. Normalization (Up to 3NF)

- Eliminates data redundancy (no duplicate storage of product details, user info, etc.)
- Reduces storage space and improves query efficiency
- Simplifies maintenance—updating a product name requires only one database change
- Prevents update, insert, and delete anomalies

3. Performance Optimization

- Strategic indexing on frequently queried columns (user_id, product_id, order_id, email, sku)
- Composite indexes on combined filters (user_id + product_id in shopping cart)
- Indexes on date columns for range queries and reporting
- Optimized for complex joins without table scans

4. Scalability

- Table partitioning potential based on order_date for historical data
- Separate tables for high-volume data (OrderItems, OrderShipment, Reviews)
- Efficient handling of growing catalog and user base
- Support for sharding strategies if needed

5. Business Logic Support

- Tracks complete order lifecycle (Pending → Confirmed → Processing → Shipped → Delivered)
- Supports multiple payment methods and payment status tracking
- Enables inventory management with reorder levels
- Supports admin role-based access control with granular permissions
- Tracks verified purchases to ensure authentic reviews

6. Reporting & Analytics

- Normalized structure enables complex aggregate queries
- Date tracking enables time-based analytics (monthly sales, trends)
- Separate review and wishlist tables allow customer behavior analysis
- Support for KPI calculations (revenue, average order value, product performance)

7. Real-World Features

- Multiple delivery addresses per user (for convenience)
- Shopping cart with temporary storage (not yet committed orders)
- Shipment tracking integration with carriers
- Review verification (is_verified_purchase flag ensures authentic feedback)
- Admin permission system with granular control

8. Data Security

- Foreign key constraints prevent orphaned records
- ON DELETE CASCADE/RESTRICT ensures proper data cleanup
- Password stored as hash (not plaintext)
- Role-based access control (RBAC) for admin functions
- JSONB field for flexible payment gateway responses without schema changes

9. Flexibility & Extensibility

- JSONB field in payments table allows storing diverse payment gateway responses
- Enum types can be extended without structural changes
- Parent category support for hierarchical product categories
- Easy to add new address types, admin permissions, or payment methods

10. Realistic E-Commerce Operations

- Supports multi-warehouse inventory management
- Tracks both selling price and cost price (for profitability analysis)
- Manages product lifecycle (active/inactive status)

- Supports promotional features (discounts, shipping costs)
- Integrates with shipping carriers via tracking numbers

Key Design Decisions Explained

Why separate OrderItems from Orders?

- Maintains 2NF/3NF (price and quantity are specific to each order, not the product)
- Allows historical pricing records (product price might change; order items preserve original price)
- Enables efficient reporting on line items

Why use JSONB for payment gateway response?

- Different payment gateways return different response formats
- Provides flexibility without schema changes
- Can query JSON data in PostgreSQL with operators

Why not store user details in Orders?

- Violates 3NF (transitive dependency: order_id → user_id → user details)
- Causes data redundancy if user updates their info
- Foreign key reference ensures consistency

Why Inventory as separate table?

- Separates product metadata from stock management
- One-to-one relationship with products
- Allows real-time inventory queries without product table scans

Why track both unit_price and line_total in OrderItems?

- Redundancy is acceptable here for denormalization (improves query speed)
- Preserves data integrity for financial records
- Speeds up order summary calculations

This design is production-ready, follows database best practices, and handles real-world e-commerce scenarios efficiently!